Requirements Traceability Matrix

Id	Requirement	Related Use Case	Fulfilled By	Test	Description
1	Create User Profile	UC 6	profileForm && UserProfile && userManager::addProfile() && ScreenManager::setUpProfileScreen && ScreenManager::showCreateProfileScren	User goes from either the first or home screen to create a new user, and afterwards can see if it was successfully created	The user can create a new profile, assuming there are less than 5 total. They do this by inputting the required information, such as name, height, email, sex, and more
2	Delete User Profile	UC 1	userManager::removeProfile() && ScreenManager::showEditScreen() && editForm::profileDeleted	Create two users and delete them; the first should send us to the home screen, the second to the first screen	Unless there are no profiles to delete, the user is allowed to delete profiles from their edit screens, removing them from the system. If the removed profile was the final one, the user is sent to the first screen, and otherwise the home screen
3	Login	UC 2	LoginForm && ScreenManager::showLoginScreen() && userManager::setCurrentUser() && homeScreen::swapProfileClicked	Create two users, and try to log into the first one. First input an incorrect password, which should throw an error. Then input the correct one, and it should fully log in	The user goes into a logged out profile and enters their password. If it's correct, the user has logged into the chosen profile, and logged out of the previous one
4	Edit users	UC 1	ScreenManager::showEditScreen && userManager::editProfile() && HomeScreen::profil	Create at least 1 user, go in, and edit anything wanted. Click	The user first inputs the profile's password, and is then allowed to change any of the

			eSelected && editForm	confirm, and check to see that the details are indeed updated	existing profile details, including adding neglected optional ones.
5	Log out	UC 1	userManager::clearCurrentUser() && HomeScreen::LogOutProfileClicked() && screenManager::setUpHomeScreem	When logged into a user, click the log out button. This should make the user logged out, and restrict certain activities, like measurements and histories	The user logging out is very simple, they just click the “Log Out” button next to the logged in user, and the screen refreshes, showing no logged in user, and all users available to log into.
6	Taking User Measurement	UC 3	MeasurementForm && MeasurementManager && screenManager::showMeasureScreen() && HomeScreen::measureNowClicked	Click on the “Measure Now” button, and go through the simulation. There should be a delay in continuing as the simulation simulates moving the device to different locations. Afterwards, click the “View Metrics” button to get a visualization of the metrics	When taking measurements, the user is asked to press the next button after lifting the device of the skin, which is simulated for the user. After taking all of the measurements, the user is told as such and given the option to see a visualization of the data, which is also available afterwards in the history section
7	Users Accessing Old Measurements	UC 5	historyForm && healthMetricVisualization && ScreenManager::showHistoryScreen() && HomeScreen::historyClicked	Going from the home screen to the history form, and checking to see that all the recent measurements are there and that the user can see a visualization of them	When clicking on the History button on the Home Screen, the user is shown old measurements, and can look at their visualizations

8	Allowing for an arbitrary number of users	N/A	MAX_PROFILES being a single editable variable	Changing MAX_PROFILES to a different number should allow for a different maximum, with minimal issues	The max number of users can be changed if needed by changing what max number is checked
9	Recharge Battery	UC 4	homescreen::chargeClicked() && screenmanager::setUpHomeScreem && Battery::deplete()	Deplete battery and click “Charge Battery” button on homescreen	When clicking the button, the screenmanager depletes the battery by -100 + the current battery charge
10	Get Specialist Recommendations		homescreen::specialistClicked() && screenmanager::setUpHomeScreem	Click “Specialist Recommendations” button on homescreen	When clicking the button, the screen manager will throw up a placeholder saying that there isn’t any recommendations
11	Storing User Profile	UC 6	UserManager::saveProfiles() && UserManager::loadProfiles	Normal app use, including exiting the application and re-entering	The user profiles are persistent, and stored in a json file. In the simulation, they are stored in a QVector, and there are function to load and save them to and from file.
12	Storing User Measurements	UC 3	MeasurementManager::storeMeasurements() && storageManager::saveMeasurement()	Take measurements, and check if they’re saved on the corresponding external file	After the testing is finished, the measurement manager organizes the readings under a JsonArray and the storage manager stores it in a file named after the current time

13	Shutdown at 5% battery	UC 4	Battery::sendShutdown() && screenmanager:: setUpHomeScreem && ScreenManager:: ScreenManager()	Depleting Battery until reaching 5%, and attempting to record measurements	When the battery is at 5% or less, the user is not allowed to take measurements, and is taken to the home screen if currently taking them
14	Battery Depletes	UC 4	Battery::deplete() && Battery:: depleteOne()	Wait the 15 to see if it depletes over time, and take measurements to see if it depletes then	The battery depletes when recording measurements, and passively every 15(?) seconds
15	Low Battery Warning	UC 4	Battery:: sendLowPower() && ScreenManager:: ScreenManager()	Deplete the battery until less than 20%, and see if the warning shows up	The battery gives a low battery warning when at less than 20%

Use Cases

Use Case 1: User Profile Management

Primary Actor(s): User
Stakeholders:
- User
- System Administrator

Pre-condition(s):
- Application is running
- User has access to the system

Success Guarantee(s):
- User profiles can be created, edited, and managed
- Profiles are securely stored

Main Success Scenario:
1. User opens the application
2. User views existing profiles
3. User can create a new profile
4. User can edit an existing profile
5. User can delete a non-current profile
6. User can switch between different profiles

Extensions:
5a. If user tries to delete the current profile, system prevents deletion
5b. If only one profile exists, deletion requires confirmation

Use Case 2: User Login

Primary Actor(s): User
Stakeholders:
- User
- System Security

Pre-condition(s):
- User profile exists
- User has email and password

Success Guarantee(s):
- Successful login grants access to user's profile and features
- Failed login prevents unauthorized access

Main Success Scenario:
1. User selects a profile
2. User enters password
3. System validates credentials
4. User gains access to profile-specific features

Extensions:
2a. User can go back to profile selection
3a. If password is incorrect, show error message
4a. If login is successful, the system sets the current user profile, refreshes the Home Screen.

Use Case 3: Health Measurement

Primary Actor(s): User
Stakeholders:
- User
- Healthcare Professionals

Pre-condition(s):
- User is logged in
- Device is charged
- User follows measurement instructions
- Battery is not in critical state

Success Guarantee(s):
- Accurate measurements are captured
- Measurements are stored securely
- User receives visualization of results

Main Success Scenario:
1. User selects "Measure Now"
2. System provides measurement instructions
3. User follows step-by-step measurement process
4. System captures measurements at specified points
5. System generates metric visualization
6. Measurements are stored in user's history

Extensions:
1a. If user is not logged in, system shows warning and prevents measurement
3a. If battery is low, prevent measurement
5a. Allow user to view historical measurements

Use Case 4: Battery Management

Primary Actor(s): User
Stakeholders:
- User
- Device Manufacturer

Pre-condition(s):
- Device is powered on
- Battery tracking is active

Success Guarantee(s):
- Battery status is accurately tracked
- User is warned of low battery
- System prevents actions when battery is critically low

Main Success Scenario:
1. System continuously monitors battery level
2. Display battery percentage
3. Warn user when battery reaches low power state
4. Prevent measurements when battery is critically low
5. Provide battery charging functionality

Extensions:
5a. When battery is critically low, shows warning, and returns to home screen

Use Case 5: Measurement History Management

Primary Actor(s): User
Stakeholders:
- User
- Healthcare Professionals

Pre-condition(s):
- User is logged in
- Measurements have been previously taken

Success Guarantee(s):
- All user measurements are retrievable
- Measurements can be visualized
- Historical data is securely stored

Main Success Scenario:
1. User selects "History"
2. System displays table of historical measurements
3. User can select specific measurements
4. System generates detailed visualization

Extensions:
1a. If user is not logged in, system shows warning and prevents access to history
2a. Filter measurements by date

Use Case 6: Create Profile

Name: Create User Profile
Primary Actor(s): User
Stakeholders:
- User
- System Administrator

Pre-condition(s):
- Application is running
- Maximum number of profiles has not been reached
- No duplicate profiles exist

Success Guarantee(s):
- New user profile is created successfully
- Profile is stored securely in the system
- User can immediately use the new profile

Main Success Scenario:
1. User selects "Create Profile" option
2. System presents profile creation form
3. User enters required profile information
4. System validates entered information
5. System creates and saves new profile
6. New profile becomes available for selection and login

Extensions:
1a. If maximum profiles reached, system prevents profile creation and shows warning

2a. Provide option to cancel profile creation
4a. If required fields are incomplete, prevent profile creation
4b. Prevent creation of profile with wrong email
4c. Validate email and password formats

Here's the revised Use Case for Screen Manager with the Use Case Flow integrated into the Main Scenario and Extensions:

Use Case 7: Screen Management

Primary Actor: User

Stakeholders:
- User
- System Administrator

Pre-conditions:
- Application is running
- QStackedWidget is initialized
- Battery object is initialized
- Maximum number of profiles has not been reached

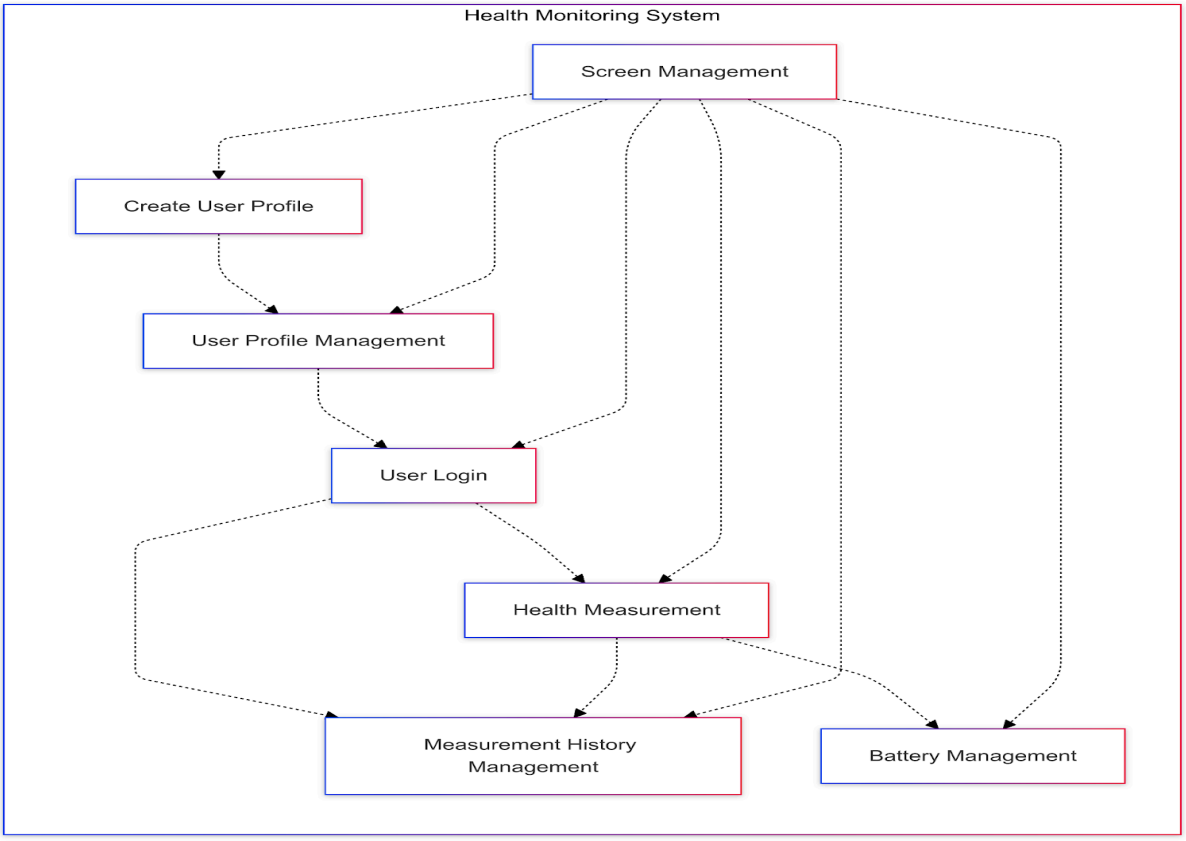
Success Guarantees:
- Screens are properly managed and displayed
- User can navigate between different screens seamlessly
- All screens are properly initialized and displayed
- User can navigate between screens without errors

Main Success Scenario:
1. System initializes ScreenManager with QStackedWidget
2. ScreenManager sets up initial screens (First Screen, Profile Screen, Home Screen)
3. User interacts with the application, triggering screen changes
4. ScreenManager handles screen transitions and updates
5. User or system triggers `showFirstScreen()`
6. ScreenManager displays the initial screen with options to create profile or enter
7. User selects "Create Profile"
8. ScreenManager calls `showCreateProfileScreen()`
9. User fills out profile information
10. On save, ScreenManager refreshes Home Screen and navigates to it
11. User selects a profile
12. ScreenManager calls `showLoginScreen(profileIndex)`
13. User enters credentials
14. On successful login, ScreenManager shows Edit Screen or swaps to Home Screen
15. User clicks "Measure Now" on Home Screen
16. ScreenManager calls `showMeasureScreen()`
17. User performs health measurement
18. On completion, ScreenManager returns to Home Screen
19. User clicks "History" on Home Screen
20. ScreenManager calls `showHistoryScreen()`

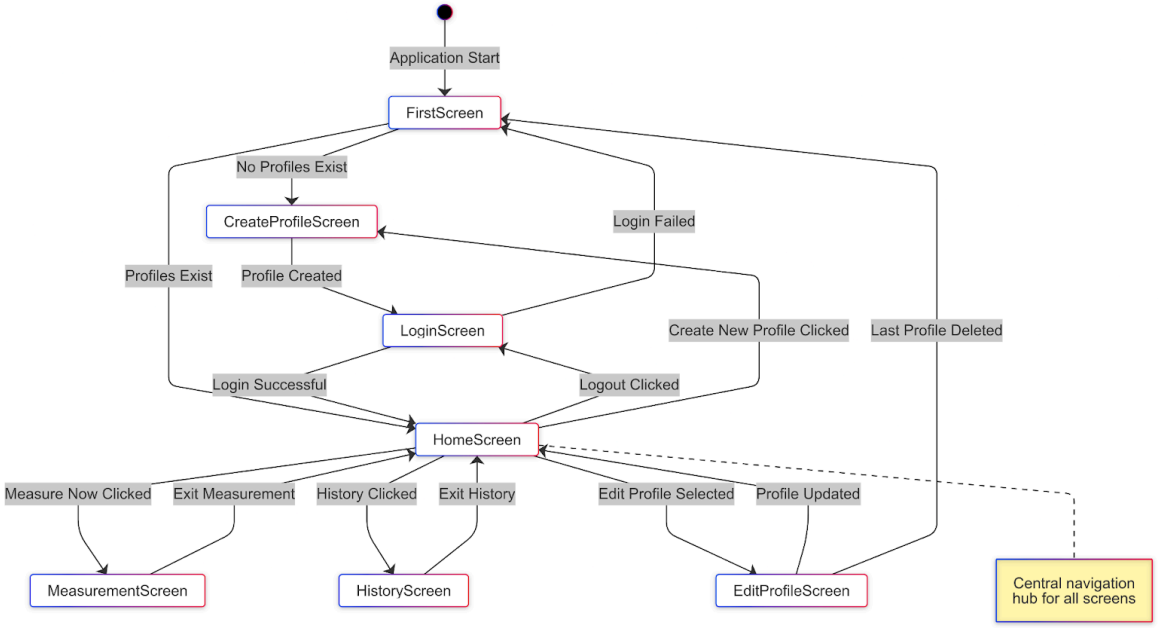
21. User views measurement history
22. On exit, ScreenManager returns to Home Screen
23. ScreenManager continuously monitors and displays battery level
24. ScreenManager prevents certain actions when battery is critical
25. ScreenManager allows user to recharge battery
26. ScreenManager calls refreshHomeScreen() when needed
27. ScreenManager updates Home Screen with current user information, profiles, and battery status
28. ScreenManager manages memory by cleaning up unused screens

Extensions:
2a. If no profiles exist, system prevents entering and shows warning
3a. If maximum profiles reached, system prevents profile creation and shows warning
4a. If login fails, system shows error and allows retry
5a. If last profile is deleted, system returns to First Screen
6a. If no profiles exist, system prevents entering and shows warning
8a. If battery reaches shutdown level, system forces return to Home Screen
14a. After login, ScreenManager calls `showEditScreen(profileIndex)`
14b. User can edit or delete profile
14c. ScreenManager updates and refreshes Home Screen accordingly
15a. If user is not logged in or battery is critical, system shows warning and prevents measurement
19a. If user is not logged in, system prevents access to History screen
23a. When battery is critically low, system forces shutdown, shows warning, and returns to home screen
27a. ScreenManager updates battery status on all relevant screens

Use Case Diagram



State machine for ScreenManager:



Design Explanation:

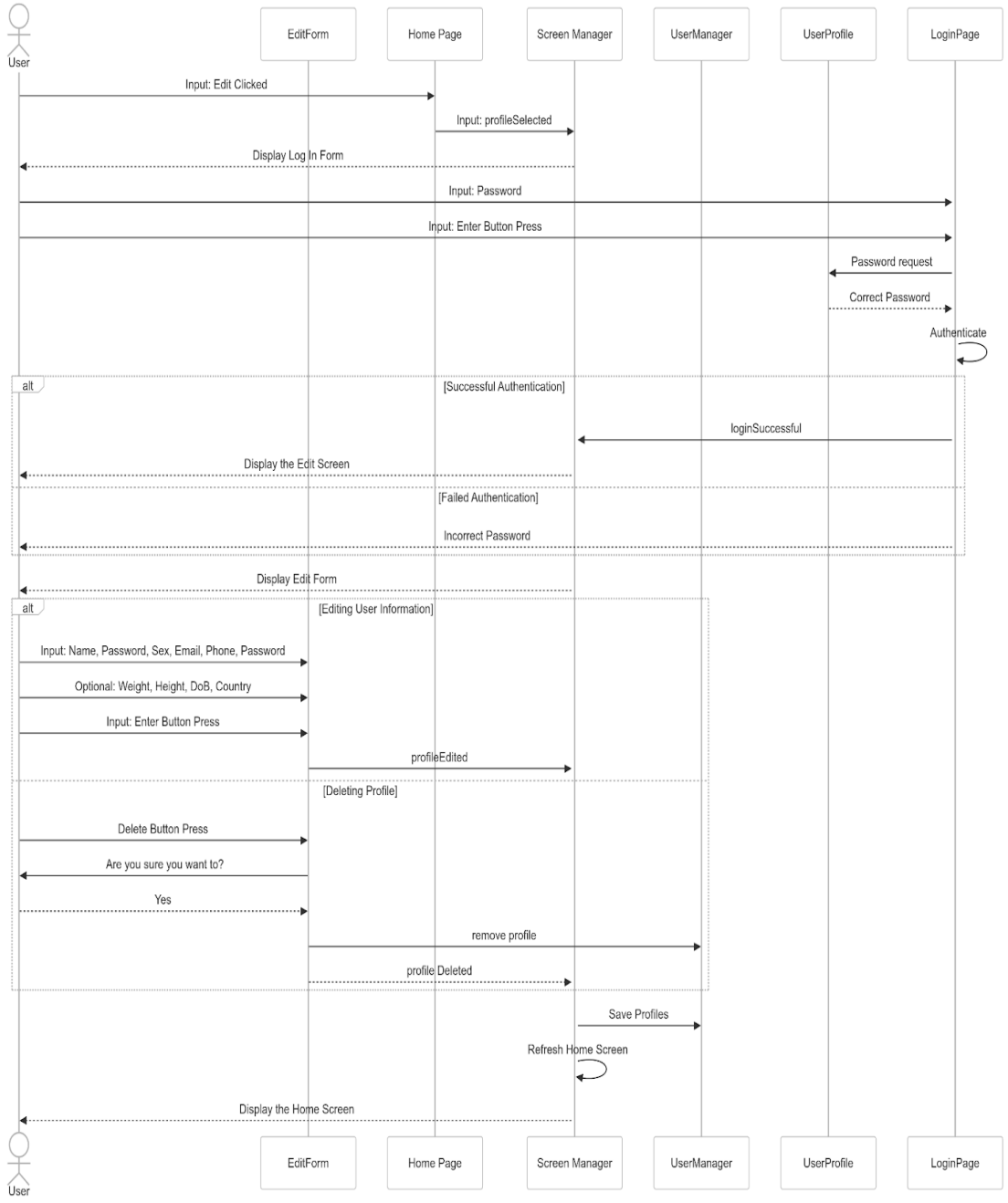
In designing the Radotech Application , our primary focus was on creating a modular, flexible system for managing health measurements and user interactions. The core of the system is the ScreenManager, a controller that manages the transitions between various screens like HomeScreen, LoginScreen, and MeasurementScreen. This central component works closely with the UserManager, which handles the management of user profiles, enabling creation, editing, and tracking of user data.

The application's data persistence is managed by the StorageManager, a simple class responsible for saving and loading user profiles, measurements, and configurations using JSON serialization. The MeasurementManager complements this by providing measurement capturing, validation, and analysis capabilities, ensuring accurate health data processing.
A unique aspect of the design is the Battery class, which integrates directly with the ScreenManager to provide real-time battery status and power state notifications. This design ensures that device health is continuously monitored and displayed to the user. The overall architecture emphasizes loose coupling between components, with each class having a distinct, well-defined responsibility.

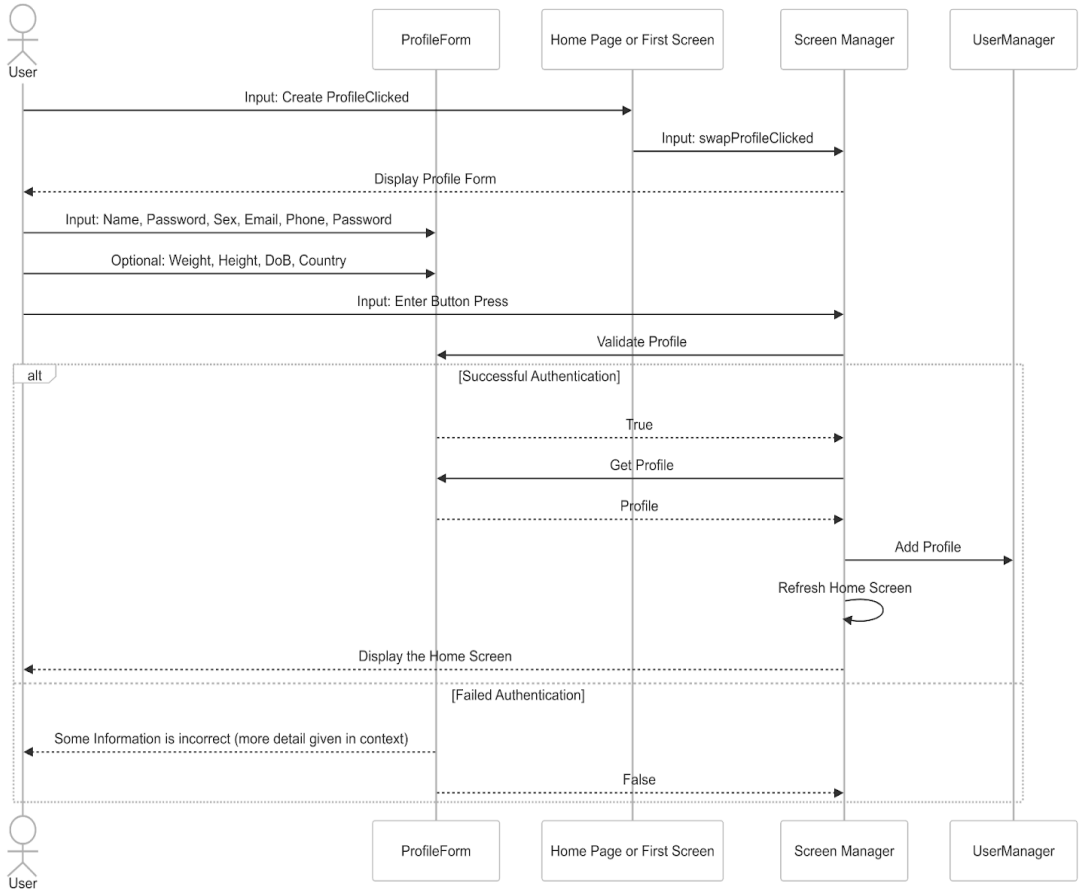
The interaction flow is straightforward: the application initializes through the ScreenManager, which checks existing user profiles and displays the appropriate initial screen. From there, users can navigate through various screens, with the system coordinating data management, user authentication, and measurement tracking seamlessly. This approach creates a flexible, user-friendly application that prioritizes modularity, ease of navigation, and comprehensive data handling.

Sequence Diagrams

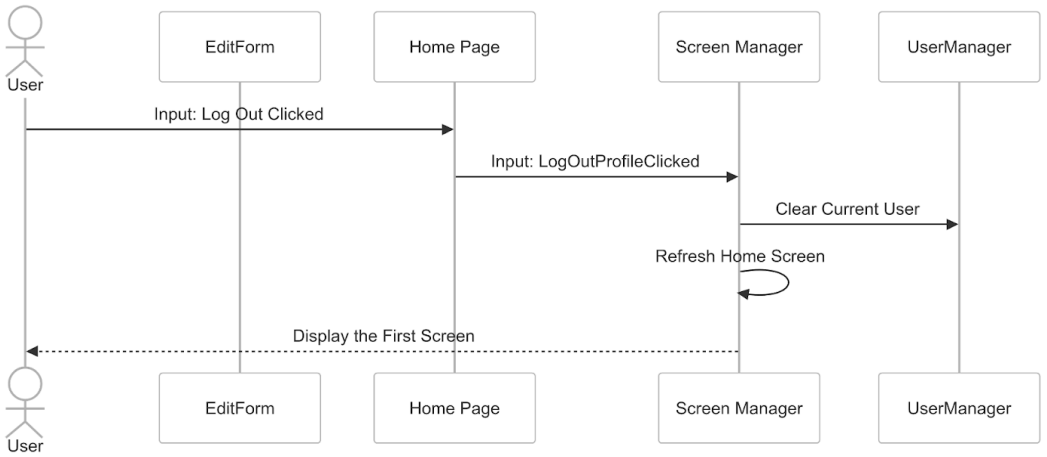
1. Edit & Deleting Profile:



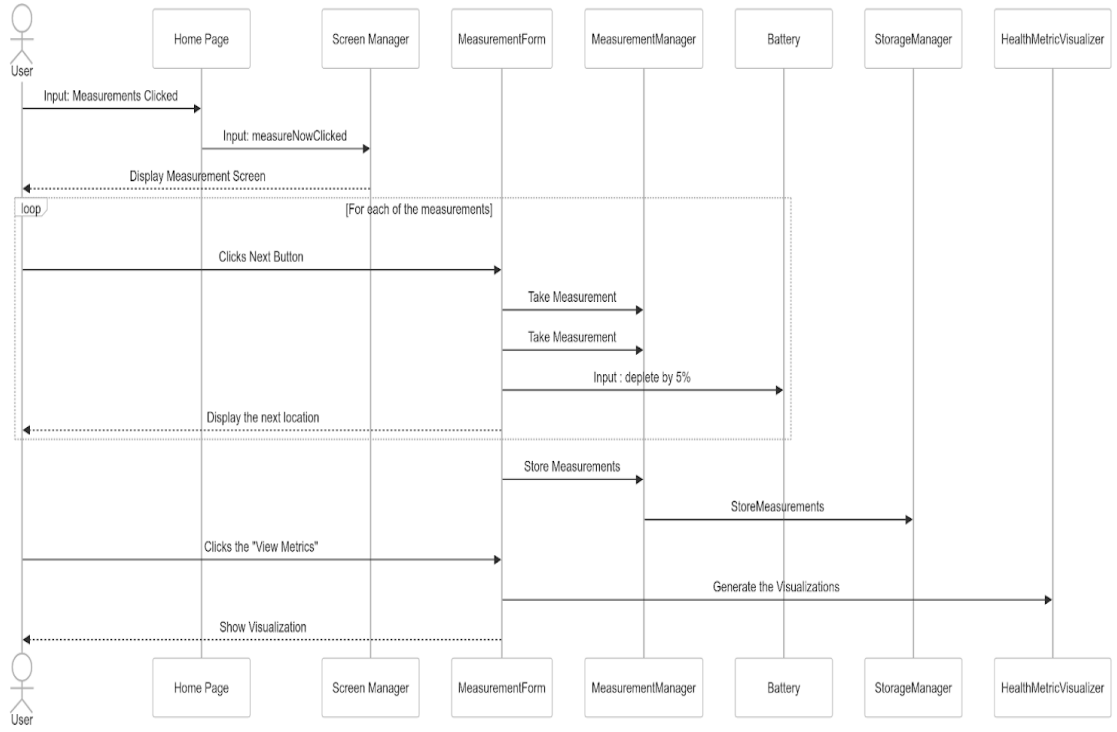
2. Creating Profile



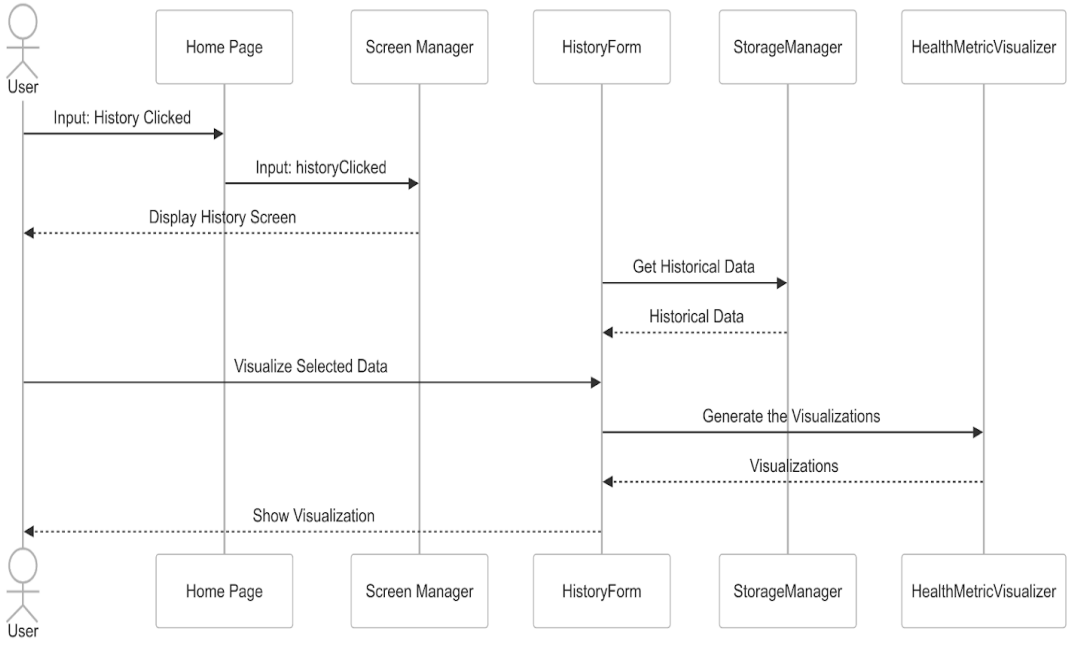
3. Logging Out



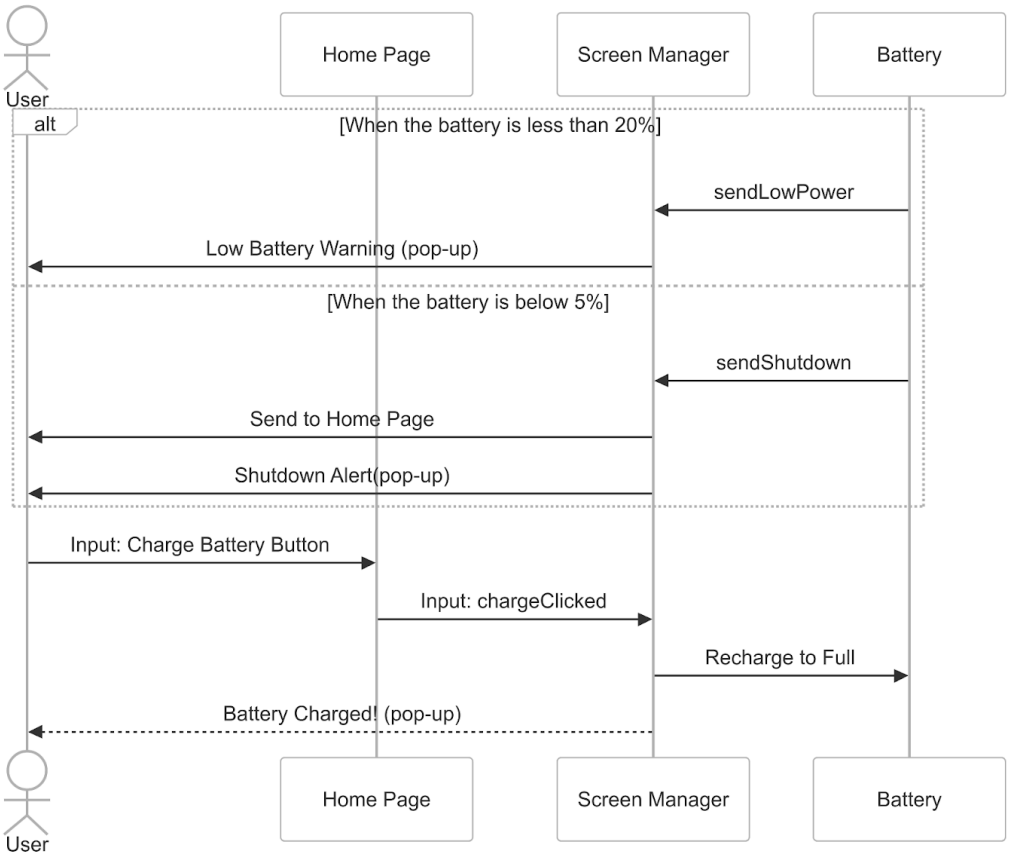
4. Taking Measurements



5. Viewing Historical Data



6. Battery Warnings and Chargings



7. Viewing Specialist Recommendations

